Universidad de San Carlos de Guatemala
Escuela de Ciencias y Sistemas
Facultad de Ingeniería
Organización de lenguajes y compiladores 2



Proyecto 2

Manual Técnico

Andrea Nicte Vicente Campos
Segundo Semestre 2020
Carnet: 201404104

Guatemala 18 de Mayo de 2020

Expresiones regulares

INTEGER	[0-9]+
DOUBLE	[0-9]+"."[0-9]+
IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*

Precedencia utilizada

A continuación podemos observar que la precedencia más baja la tienen las operaciones lógicas. Siendo la más baja de ellas or y la más alta el grupo de “mayor que”, “menor que”, “mayor e igual que” y “menor e igual que”.

No.	Presedencia	Simbolo
1	Izquierda	‘++’ ‘--’
2	Izquierda	‘^’
3	Izquierda	'||'
4	Izquierda	'&&'
5	Izquierda	‘==' '!=' '==='
6	Izquierda	‘<' '>' '<=' '>='

Seguidas por las operaciones aritméticas, en las cuales la suma y la resta poseen la precedencia más baja, seguidas por la multiplicación, división y módulo. Siendo la ^^ la más alta precedencia

7	Izquierda	'+' '-'
8	Izquierda	'*' '/' '%'
9	Izquierda	'^^'
10	Derecha	‘-' Unario

11	Derecha	'strc'
12	Izquierda	“!'

Símbolos Terminales

#	Símbolo Terminal	Explicación
1	"print"	Imprime valores y/o expresiones en consola
2	"as"	Indica los parámetros de una definición
3	"define"	Establece un nuevo elemento u objeto
4	"strc"	Establece una estructura
5	"break"	Interrumpe el proceso en curso
6	"boolean"	Tipo de dato booleano
7	"case"	Utilizada para la sentencia switch case
8	"char"	Tipo de dato caracter
9	"continue"	Reanuda un proceso
10	"default"	Conjunto de acciones por defecto en la sentencia switch case
11	"double"	Tipo de dato double
12	"do"	Utilizado para la sentencia do while
13	"else"	Utilizado para las sentencias if else
14	"false"	Valor falso en variables booleanas
15	"for"	Utilizado para el ciclo for
16	"if"	Utilizado para la condición de sentencias if else
17	"import"	Utilizado para importar funciones
18	"integer"	Tipo de dato integer

19	"null"	Valor null en cualquier variable
20	"return"	Retorna un valor
21	"switch"	Utilizada para la sentencia switch case
22	"str"	
23	"throw"	Lanza automáticamente una excepción
24	"true"	Valor verdadero para una variable booleana
25	"void"	Función que no retorna ninguna expresión
26	"while"	Palabra reservada para ciclos while y do while
27	"var"	Define variables
28	"const"	Define constantes
29	"global"	Define globales
30	"{"	Llave de apertura
31	"}"	Llave de cierre
32	"[]"	Acceso a elemento de arreglo
33	"["	Corchete de apertura
34	"]"	Corchete de cierre
35	";"	Punto y coma
36	":="	Palabra reservada necesaria para declarar estructuras
37	":"	Dos puntos, utilizado en case y otras sentencias
39	","	coma, utilizada en declaración de arreglos y otras sentencias
40	"."	punto, utilizado en llamadas y otras sentencias
41	"||"	Operador lógico or
42	"&&"	Operador lógico and
43	"!="	Operador lógico desigualdad
44	"!"	Negación unaria
45	"*"	Aritmético multiplicación
46	"/"	Aritmético división
47	"--"	Decremento

48	"-"	Aritmético resta
49	"++"	Incremento
50	"+"	Aritmético suma, concatenación de cadenas
51	"^^"	Aritmético potencia
52	"^"	Operador lógico xor
53	"!"	Not unario
54	"%"	Multiplicativa modular
55	"("	Paréntesis de apertura
56	")"	Paréntesis de cierre
57	"==="	Operador de igualdad que devuelve true si los operandos tienen la misma dirección de memoria
58	"=="	Relacional de igualdad
59	"="	Definición de igualdad
60	"<="	Relacional de menor o igual que
61	">="	Relacional de mayor o igual que
62	"<"	Relacional menor que
63	">"	Relacional mayor que
64	"@"	

Símbolos no terminales

#	Símbolo NO Terminal	Explicación
1	LISTA_SENTENCIAS_GLOBALES	Inicio gramatica
2	SENTENCIAS_GLOBALES	Símbolo no terminal para completar lista
3	SENTENCIAS_GLOBALE	Sentencia a nivel global individual
4	DECLARACION_VARIABLES_GLOBALES	Declaración de variables a nivel de global
5	DECLARACION_FUNCIONES	Declaración de variables a nivel de función
6	DECLARACION_STRUCTS	Declaración de estructuras
7	DECLARACION_ESPECIALES	Declaraciones utilizando ‘var’ ‘global’ o ‘const’
8	SENTENICAS_DE_FUNCION	Lista de funciones
9	SENTENCIAS_FUNCION	Símbolo no terminal para completar lista
10	SENTENCIA_FUNCION	Sentencia de función
11	ASIGNACION	Asignación
12	DECLARACION_VARIABLE_LOCAL	Declaración de variables a nivel función
13	ICREMENTO	Sentencia de incremento
14	INVOCACION_FUNCIONES	Invocación regular
15	INVOCACION_PUNTO	Invocación de funciones con ‘.’
17	INSTRUCCION_WHILE	Ciclo while
18	INSTRUCCION_FOR	Ciclo for
19	INSTRUCCION_DOWHILE	Ciclo do while

20	INSTRUCCION_IF	Sentencia if
21	INSTRUCCION_SWITCH	Sentencia switch
22	RETURN	Sentencia para return
23	BREAK	Sentencia para break
24	CONTINUE	Sentencia para continue
26	importSentence	Sentencia de import
27	imports	Detalle del import
28	ATRIBUTOS_STRUCT	Símbolo no terminal para completar lista atributos
29	ATRIBUTOS_STRUCT	Definición de atributos
30	DECLARACION_VARIABLES_GLOBALES	Declaración de variables a nivel global
31	DECLARACION_VARIABLE_LOCAL	Declaración de variables a nivel función
32	DECLARACION_VARIABLE	Símbolo no terminal para completar lista
33	DEC_VARIABLE	Declaración de variables
34	TIPO	Tipos de datos
35	ELEMENTOS_ARREGLOList	Lista de elementos array
36	ELEMENTOS_ARREGLOS	Símbolo no terminal para completar lista
37	ELEMENTOS_ARREGLO	Elementos array
38	LISTA_PARAMETROS	Declaración de lista de parámetros
39	PARAMETROS	Símbolo no terminal para completar lista
40	PARAMETRO	Parámetro de lista
41	SENTENCIA_FUNCION	Lista de sentencias para función
42	TIPO_PRIMITIVO	Tipos primitivos
43	ASIGNACION	Asignación
44	CASTEO_EXPLICITO	Casteo de diferentes tipos de datos
45	LISTA_EXPRESIONES	Lista de expresiones
46	expressions	Símbolo no terminal para completar lista
47	expression	Expresiones posibles dentro del lenguaje

48	CUERPO_CASE	Case block
49	CUERPO_IF	Complementos de la sentencia if else

```
Gramatica:
Start1
    : Start
    ;

LISTA_SENTENCIAS_GLOBALES
    : SENTENCIAS_GLOBALES
    |
    ;
SENTENCIAS_GLOBALES
    : SENTENCIAS_GLOBALES SENTENCIAS_GLOBALE
    | SENTENCIAS_GLOBALE
    | IMPORT ';'
    ;
SENTENCIAS_GLOBALE
    : DECLARACION_VARIABLES_GLOBALES ';'
    | DECLARACION_FUNCIONES
    | DECLARACION_STRUCTS ';'
    | DECLARACION_ESPECIALES ';'
    ;

SENTENICAS_DE_FUNCION
    :
    | SENTENCIAS_FUNCION
    ;
SENTENCIAS_FUNCION
    : SENTENCIAS_FUNCION SENTENCIA_FUNCION
    | SENTENCIA_FUNCION
    ;
SENTENCIA_FUNCION
    : ASIGNACION ';'
    | DECLARACION_VARIABLE_LOCAL ';'
    | ICREMENTO ';'
    | INVOCACION_FUNCIONES ';'
    | INVOCACION_PUNTO ';'
    | INSTRUCCION_WHILE
    | INSTRUCCION_FOR
    | INSTRUCCION_DOWHILE
    | INSTRUCCION_IF
    | INSTRUCCION_SWITCH
    | RETURN ';'
    | BREAK  ';'
    | CONTINUE ';'
    | DECLARACION_STRUCTS ';'
    | DECLARACION_ESPECIALES  ';'
    ;

{SENTENCIAS}

IMPORT
    : 'import' imports
    ;
imports
    : imports ',' PATH
    | PATH
```

```
    ;
PATH
    : STRING
    ;

DECLARACION_STRUCTS
    : 'define' IDENTIFIER 'as' '[' ATRIBUTOS_STRUCT ']'
    ;
ATRIBUTOS_STRUCT
    : ATRIBUTOS_STRUCT ',' ATRIBUTO_STRUCT
    | ATRIBUTO_STRUCT
    ;
ATRIBUTO_STRUCT
    : TIPO IDENTIFIER '=' expression
    | TIPO IDENTIFIER
    | TIPO '[' ']' IDENTIFIER '=' expression
    | TIPO '[]' IDENTIFIER '=' expression
    | TIPO '[' ']' IDENTIFIER
    | TIPO '[]' IDENTIFIER
    ;


DECLARACION_VARIABLES_GLOBALES
    : TIPO DECLARACION_VARIABLE
    | TIPO '[' ']' DECLARACION_VARIABLE
    | TIPO DECLARACION_VARIABLE '=' expression
    | TIPO '[' ']' DECLARACION_VARIABLE '=' expression
    ;
DECLARACION_VARIABLE_LOCAL
    : TIPO_PRMITIVO DECLARACION_VARIABLE
    | IDENTIFIER DECLARACION_VARIABLE
    | TIPO_PRMITIVO '[' ']' DECLARACION_VARIABLE
    | TIPO_PRMITIVO '[]' DECLARACION_VARIABLE
    | IDENTIFIER '[]' DECLARACION_VARIABLE
    | TIPO_PRMITIVO DECLARACION_VARIABLE '=' expression
    | IDENTIFIER DECLARACION_VARIABLE '=' expression
    | TIPO_PRMITIVO '[' ']' DECLARACION_VARIABLE '=' expression
    | TIPO_PRMITIVO '[]' DECLARACION_VARIABLE '=' expression
    | IDENTIFIER '[]' DECLARACION_VARIABLE '=' expression
    ;

DECLARACION_ESPECIALES
    : 'var' IDENTIFIER ':=' expression
    | 'const' IDENTIFIER ':=' expression
    | 'global' IDENTIFIER ':=' expression

    ;

DECLARACION_VARIABLE
    : DECLARACION_VARIABLE ',' DEC_VARIABLE
    | DEC_VARIABLE
    ;
DEC_VARIABLE
    : IDENTIFIER
    ;

ARRAY_INIT
    : '{' ELEMENTOS_ARREGLOList '}'
```

```
    ;

ELEMENTOS_ARREGLOList
    :
    | ELEMENTOS_ARREGLOS
    ;
ELEMENTOS_ARREGLOS
    : ELEMENTOS_ARREGLOS ',' ELEMENTOS_ARREGLO
    | ELEMENTOS_ARREGLO
    ;

ELEMENTOS_ARREGLO
    : expression
    | ARRAY_INIT
    ;

DECLARACION_FUNCIONES
    : TIPO  IDENTIFIER '(' LISTA_PARAMETROS ')'
      '{' SENTENICAS_DE_FUNCION '}'

    | TIPO '[' ']' IDENTIFIER '(' LISTA_PARAMETROS ')'
      '{' SENTENICAS_DE_FUNCION '}'

    | TIPO '[]' IDENTIFIER '(' LISTA_PARAMETROS ')'
      '{' SENTENICAS_DE_FUNCION '}'

    ;

LISTA_PARAMETROS
    : PARAMETROS
    |
    ;
PARAMETROS
    : PARAMETROS ',' PARAMETRO
    | PARAMETRO
    ;
PARAMETRO
    : TIPO '[' ']' IDENTIFIER
    | TIPO '[]' IDENTIFIER
    | TIPO IDENTIFIER
    ;


TIPO
    : TIPO_PRMITIVO
    | IDENTIFIER
    ;
TIPO_PRMITIVO
    : 'integer'
    | 'double'
    | 'char'
    | 'boolean'
    | 'void'
    ;


ASIGNACION
    : expression '=' expression
```

```
    ;

expression
    : expression '-' expression
    | expression '+' expression
    | expression '*' expression
    | expression '/' expression
    | expression '%' expression
    | expression '!=' expression
    | expression '==' expression
    | expression '<' expression
    | expression '>' expression
    | expression '<=' expression
    | expression '>=' expression
    | expression '||' expression
    | expression '&&' expression
    | expression '^^' expression
    | expression '^' expression
    | expression '.' IDENTIFIER
    | INVOCACION_PUNTO
    | INVOCACION_FUNCIONES
    | STRING
    | CHAR
    | DOUBLE
    | INTEGER
    | NULL
    | 'true'
    | 'false'
    | IDENTIFIER
    | expression '[' expression ']'
    | '(' expression ')'
    | '-' expression %prec UMINUS
    | '!' expression
    | expression "?" expression ":" expression
    | ICREMENTO
    | 'strc' TIPO '(' ')'
    | 'strc' TIPO '[' expression ']'
    | casting
    | expression '===' expression
    | '{' LISTA_EXPRESIONES '}'
    ;

casting
    : '(' 'integer' ')' expression %prec UINT
    | '(' 'double' ')' expression %prec UDOUBLE
    | '(' 'char'  ')' expression %prec UCHAR
    ;


INVOCACION_FUNCIONES
    : IDENTIFIER '(' LISTA_EXPRESIONES ')'
    ;


INVOCACION_PUNTO
    : expression '.' IDENTIFIER '(' LISTA_EXPRESIONES ')'
    ;
```

```
ICREMENTO
    : expression '++'
    | expression '--'
    ;


LISTA_EXPRESIONES
    : EXPRESIONES
    |
    ;
EXPRESIONES
    : EXPRESIONES ',' expression
    | expression
    ;


{SENTENICAS DE CONTROL}

INSTRUCCION_WHILE
    : 'while' '(' expression ')' '{'
        SENTENICAS_DE_FUNCION
      '}'
    ;

INSTRUCCION_DOWHILE
    : 'do' '{'
        SENTENICAS_DE_FUNCION
      '}' 'while' '(' expression ')'
    ;

INSTRUCCION_FOR
    : 'for' '(' forInit  ';' expression ';'  ICREMENTO  ')' '{'
        SENTENICAS_DE_FUNCION
      '}'
    | 'for' '(' forInit  ';' expression ';'  ASIGNACION  ')' '{'
        SENTENICAS_DE_FUNCION
      '}'
    ;

forInit
    : ASIGNACION
    | DECLARACION_VARIABLE_LOCAL
    ;

INSTRUCCION_SWITCH
    : 'switch' '(' expression  ')'   '{'
        LISTA_CASES
        'default' ':' SENTENICAS_DE_FUNCION
    '}'
    | 'switch' '(' expression  ')'   '{'
              LISTA_CASES
       '}'
    | 'switch' '(' expression  ')'   '{'

           '}'
    ;

LISTA_CASES
```

```
    : LISTA_CASES INSTRUCCION_CASE
    | INSTRUCCION_CASE
    ;

INSTRUCCION_CASE
    : 'case' expression ':' SENTENICAS_DE_FUNCION
    ;

INSTRUCCION_IF
    : 'if' '(' expression ')' CUERPO_IF
    | 'if' '(' expression ')' CUERPO_IF 'else' CUERPO_IF
    | 'if' '(' expression ')' CUERPO_IF 'else' INSTRUCCION_IF
    ;

CUERPO_IF
    : '{' SENTENICAS_DE_FUNCION '}'
    ;

RETURN
    : 'return' expression
    | 'return'
    ;

BREAK
    : 'break'
    ;

CONTINUE
    : 'continue'
    ;
```